



OpsDesk

Mini ERP + CRM Operations Portal

Technical & Product Documentation

Prepared as part of the Full Stack Developer Case Study

Frontend: <https://opsdesk-delta.vercel.app>

Backend API: <https://opsdesk-backend.onrender.com/api>

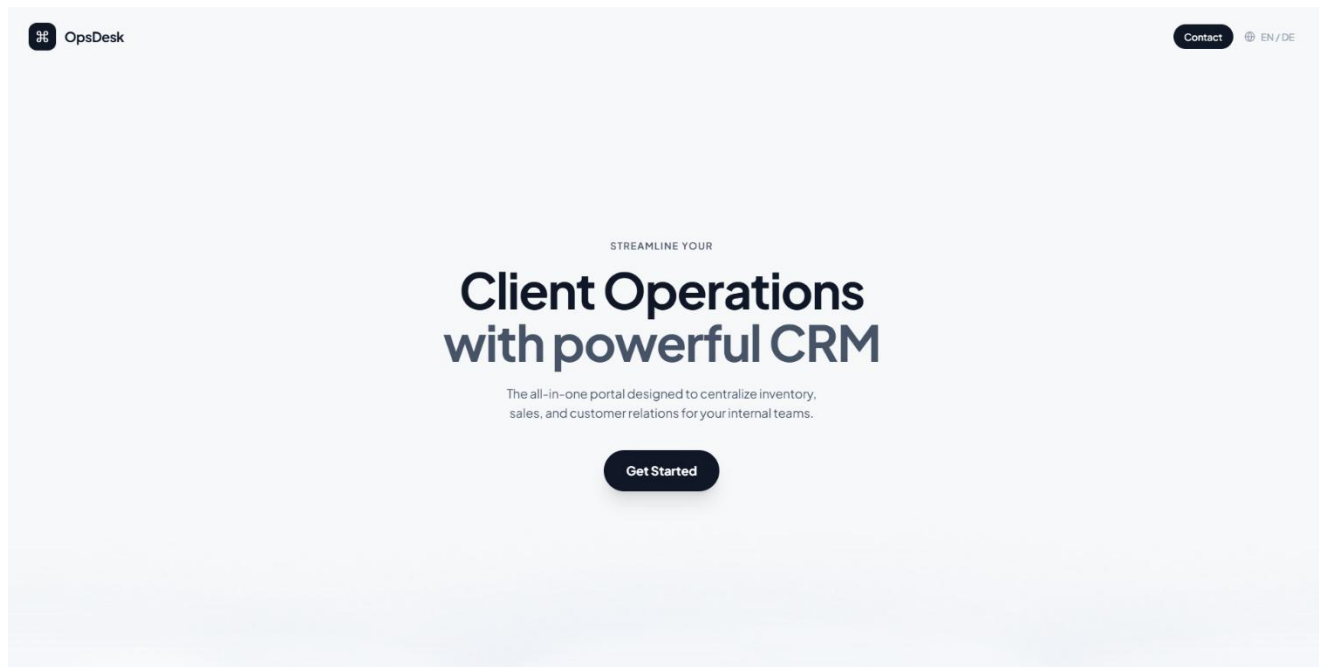


Table of Contents

Table of Contents.....	2
2. Executive Summary	3
3. Business Problem.....	3
4. Objectives & Scope	4
5. Users & Roles	4
6. Requirements.....	5
7. Functional Modules	5
8. End-to-End Business Workflow	6
9. System Architecture	6
10. Technology Stack	7
11. Frontend Architecture	8
12. Backend Architecture	9
13. Database Design	10
14. Schema / Entity Relationship Diagram	11
15. Authentication & RBAC	12
16. Business Logic & Data Integrity	13
17. API Documentation.....	13
18. Security	15
19. Testing & Quality	15
20. Docker	Error! Bookmark not defined.
21. CI/CD	17
22. Deployment Architecture	17
23. Environment Variables	17
24. Local Setup	18
25. Demo Walkthrough	18
26. Screenshots.....	19
27. Assumptions	23
28. Known Limitations	23
29. Future Enhancements.....	23
30. Engineering Decisions & Trade-offs	24
31. Conclusion.....	24

2. Executive Summary

OpsDesk is a full-stack, internal operations portal built for a wholesale/distribution business, developed to satisfy a 48-hour full-stack developer case study. It unifies customer relationship management, product and inventory control, and the sales challan (delivery note) lifecycle into a single role-aware platform used by four distinct internal teams: Admin, Sales, Warehouse, and Accounts.

Beyond the four required core modules, the project was extended with a live Reports & Analytics dashboard, a Users & Roles administration panel, a reminders/notifications system, PDF export of confirmed challans, a Dockerized deployment pipeline, and a fully custom light-mode UI/UX design system with motion detailing throughout. The application is deployed and publicly reachable end-to-end: a React frontend on Vercel, a containerized Node.js/Express API on Render, and a PostgreSQL database on Neon.

This document walks through the business problem, the engineering approach, the system design, and the operational detail of the finished product, and is intended to give a technical reviewer a complete picture of the system without needing to run it locally first.

3. Business Problem

Small and mid-sized wholesale/distribution businesses typically run their sales, warehouse, and accounts functions as three loosely-connected teams. In practice this often means each team works from its own spreadsheet or informal process — sales reps track customer conversations separately from what the warehouse knows about stock, and accounts finds out about a sale only after the fact.

This creates three recurring, costly failure modes:

- Stock counts drift out of sync with what has actually been sold, leading to overselling or wasted warehouse time double-checking availability.
- Customer follow-ups are missed because there is no shared, dated record of who needs to be contacted and when.
- Sales documents (challans/delivery notes) lose their historical accuracy if the underlying product catalog changes after the fact — a price update today shouldn't silently rewrite yesterday's invoice.

OpsDesk directly addresses these three failure modes by centralizing customer, inventory, and sales-document data behind role-based access, and by enforcing the underlying business rules — most importantly, that stock can never go negative, and that historical documents remain accurate — at the database and API layer rather than relying on manual discipline

4. Objectives & Scope

Objectives

- Demonstrate full-stack engineering competency: backend API design, relational database modeling, frontend UI/UX, and cloud deployment, within a realistic business domain.
- Implement real, testable business logic rather than simple CRUD — specifically, transactional stock control and immutable historical records.
- Ship a genuinely deployed, publicly reachable system rather than a local-only demo.

In-Scope (Required by the Case Study)

- JWT authentication with four roles: Admin, Sales, Warehouse, Accounts
- Customer CRM module (add/edit/search/detail/follow-up notes)
- Product & Inventory module with an auditable stock movement log
- Sales Challan module with Draft → Confirmed → Cancelled lifecycle and transactional stock deduction
- Clean REST APIs with validation, proper status codes, pagination and search/filter
- Environment-variable-driven configuration, documented setup, and a GitHub repository with meaningful commit history

Extended Scope (Beyond the Brief)

- Reports & Analytics dashboard (sales activity, customer breakdown, inventory health, follow-up compliance)
- Users & Roles administration module for Admin
- Reminders and a live notifications system
- Branded PDF export of confirmed challans
- Docker Compose stack (Postgres + backend + frontend) for one-command local/reproducible setup
- A fully custom, animated, light-mode design system across every screen

5. Users & Roles

OpsDesk is an internal-only system — there is no public self-registration. Accounts are provisioned by an Administrator, and every account is assigned exactly one of four roles, each scoped to the responsibilities that role actually needs.

Role	Primary Responsibility	Typical Daily Use
Admin	Full system oversight and user management	Manage users/roles, view all data, run reports, oversee operations
Sales	Customer relationships and sales documents	Add/edit customers, log follow-ups, create and confirm challans
Warehouse	Physical stock accuracy	Manage products, adjust stock, review the movement audit log
Accounts	Financial and operational visibility	View customers, challans, and the Reports dashboard (read-focused)

6. Requirements

Functional Requirements (from the Case Study Brief)

- Role-based login (Admin, Sales, Warehouse, Accounts) using JWT
- Customer CRUD with search, a detail page, and timestamped follow-up notes
- Product & Inventory CRUD with a full stock movement audit trail (IN/OUT, reason, actor, timestamp)
- Sales Challan creation with multiple line items, auto-generated challan numbers, Draft/Confirmed status, and product-snapshot storage
- Stock must never go negative; insufficient stock must return a clear API error, not a silent failure

Non-Functional Requirements

- Clean REST API design with consistent validation, HTTP status codes, and error messages
- Pagination and search/filter on list endpoints
- Responsive, professional 'admin-style' UI
- Environment-variable-driven configuration (no hardcoded secrets or URLs)
- Deployable on free-tier infrastructure without requiring the candidate to spend money

7. Functional Modules

7.1 Authentication & Roles

JWT-based login with role-aware routing on the frontend and role-enforced middleware on the backend.

Passwords are hashed with bcrypt; tokens carry the user's id and role and are verified on every protected request.

7.2 Customer CRM

Full customer record (contact details, business name, optional GST number, customer type, address, lifecycle status, follow-up date) with search, filtering, a dedicated detail page, and an append-only follow-up notes timeline.

7.3 Products & Inventory

Product catalog with SKU, category, pricing, current stock, and a configurable minimum-stock alert threshold.

Every stock change — whether a manual adjustment or the result of a confirmed challan — is written to an immutable stock movement log.

7.4 Sales Challans

The core operational workflow: a Sales user builds a challan against a customer with multiple product lines, saves it as a Draft (editable, no stock impact) or Confirms it directly (which validates and atomically deducts stock).

Confirmed challans can be cancelled, optionally restoring stock, and exported as a branded PDF.

7.5 Users & Roles (Admin)

Administrators can provision new accounts, assign roles, and activate/deactivate users without ever deleting their historical audit trail (a deactivated user's name still appears correctly on records they created in the past).

7.6 Reports & Analytics

An aggregated, date-range-filterable view for Admin and Accounts: sales activity over time, customer breakdown by type/status, inventory health, and follow-up compliance — distinct from the Dashboard's real-time, 'right now' focus.

7.7 Reminders & Notifications

Personal, per-user reminders with a scheduled date/time, surfaced through a shared notification center alongside system alerts (low stock, follow-ups due).

8. End-to-End Business Workflow

The Sales Challan lifecycle is the operational backbone of OpsDesk and is designed to mirror how a real delivery note is handled in a wholesale business:

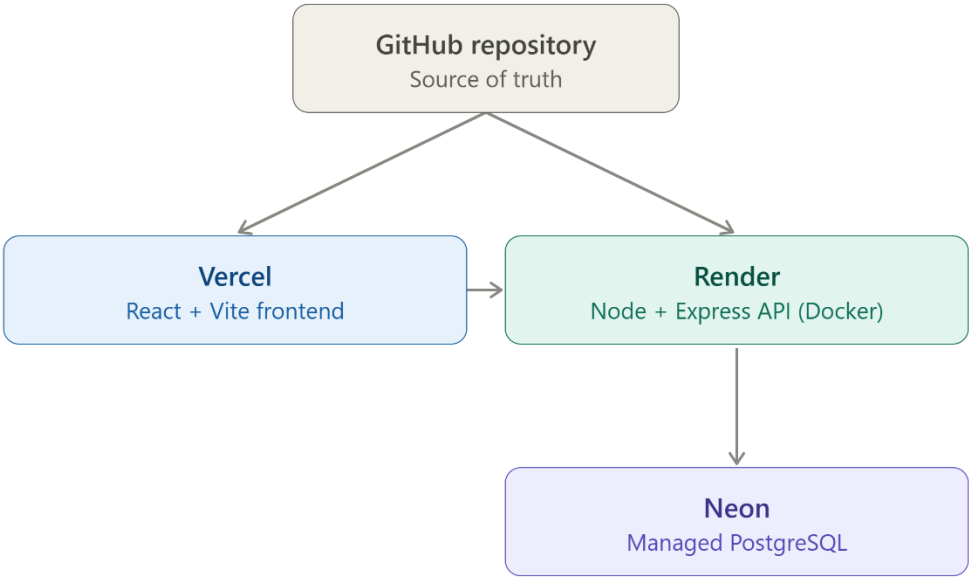
1. A Sales user selects a customer and adds one or more products with quantities.
2. The challan is saved as Draft (no stock impact, fully editable) or submitted directly as Confirmed.
3. On confirmation, available stock is validated against every line item.
4. Stock is deducted using an atomic database operation inside a single transaction — never by reading a value into application memory and writing it back.
5. A corresponding stock movement record (type OUT) is created for each line item, in the same transaction.
6. The challan is marked CONFIRMED and its product line items retain a permanent snapshot of name, SKU, and price as they were at that moment.
7. A Confirmed challan can later be Cancelled; the user may optionally choose to restore ('restock') the deducted quantities, which — again — happens atomically within one transaction alongside the status change.

Because every stock-changing step happens inside a database transaction with atomic increment/decrement operations, two challans confirming simultaneously against the same low-stock product cannot corrupt the stock count or push it negative — one succeeds, the other is cleanly rejected with a descriptive error.

9. System Architecture

OpsDesk follows a standard three-tier architecture: a React single-page application, a stateless Node.js/Express REST API, and a managed PostgreSQL database, connected over HTTPS.

Layer	Technology	Hosting
Frontend	React + TypeScript (Vite build)	Vercel
Backend API	Node.js + Express + TypeScript (Dockerized)	Render
Database	PostgreSQL (via Prisma ORM)	Neon



The frontend never talks to the database directly — all data access goes through the authenticated REST API, which is the single source of truth for validation, authorization, and business logic. This keeps the client a pure presentation layer and means the same backend could serve a different frontend (e.g. a future mobile app) without any change.

10. Technology Stack

Frontend

Purpose	Technology
Framework	React + TypeScript, built with Vite
Styling	Tailwind CSS, custom design tokens
Routing	React Router
Server state / data fetching	TanStack Query
Client state	Zustand
Forms & validation	React Hook Form + Zod
Animation	Framer Motion
Charts	Recharts
Icons	lucide-react

Backend

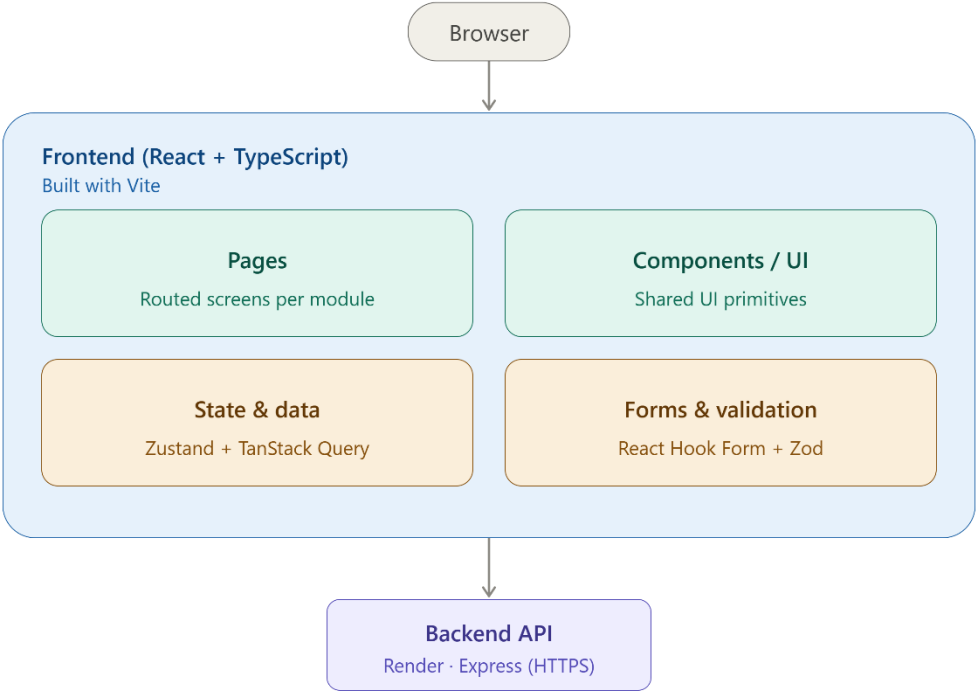
Purpose	Technology
Runtime / language	Node.js + TypeScript
Web framework	Express.js
ORM / database	Prisma ORM → PostgreSQL
Authentication	jsonwebtoken (JWT), bcrypt (password hashing)
Validation	Zod
Security middleware	Helmet, express-rate-limit, custom CORS allow-list
PDF generation	PDFKit

11. Frontend Architecture

The frontend is organized around a clear separation between reusable primitives, feature components, and routed pages:

- `components/ui/` — the shared design-system primitives: Button, Input, Select, Modal, Drawer, DataTable, StatusBadge, StatCard, Tabs, ConfirmDialog, EmptyState, plus bespoke visual components (CloudBackground, QuietCurrent wave animation, ConcentricCircles and MultiArcProgress for the Reports charts).
- `pages/` — one component per route: Login, Dashboard, CustomersList/CustomerDetail, InventoryList/ProductDetail, ChallansList/ChallanBuilder/ChallanDetail, UsersList, Reports, Profile.
- `store/` — Zustand stores for auth state and layout/UI state (e.g. sidebar collapsed/expanded).
- `hooks/` — shared logic such as `useDebounce` for search inputs.

Server data is never held in ad-hoc component state; every list and detail view is backed by a TanStack Query hook, which gives the app automatic caching, background refetching, and consistent loading/error states across every screen without repeating that logic per page. Forms uniformly pair React Hook Form with a Zod schema, so client-side validation rules are expressed the same way the backend expresses its own — reducing the chance of the two drifting apart.

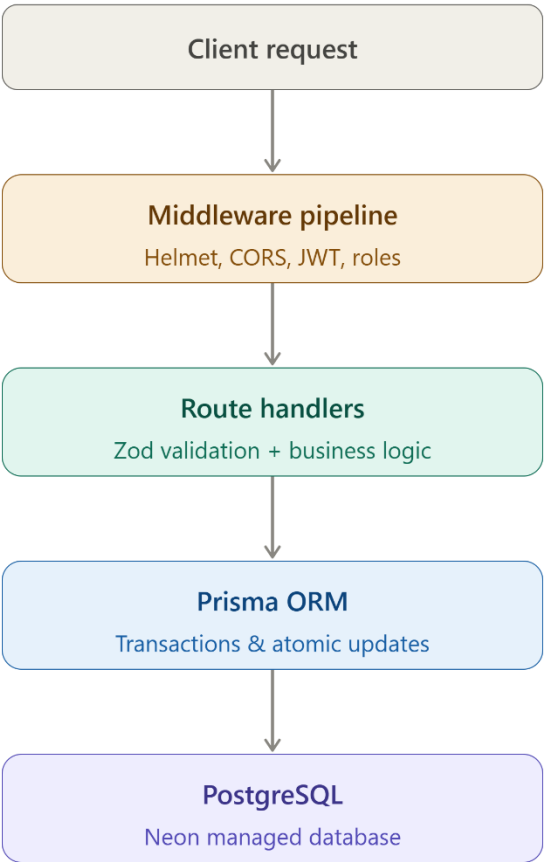


12. Backend Architecture

The backend is a modular Express application: `index.ts` bootstraps the app (security middleware, CORS, rate limiting, route mounting, and a global error handler), `db.ts` exposes a single shared Prisma client instance, and each business domain has its own router under `src/routes/`.

Module	File	Responsibility
Auth	<code>routes/auth.ts</code>	Login, current-user lookup, password change, admin seeding
Customers	<code>routes/customers.ts</code>	Customer CRUD, search/filter, follow-up notes
Products	<code>routes/products.ts</code>	Product CRUD, stock adjustment, movement history
Challans	<code>routes/challans.ts</code>	Challan CRUD, confirm, cancel, PDF export
Users	<code>routes/users.ts</code>	Admin user management
Dashboard	<code>routes/dashboard.ts</code>	Aggregated real-time KPI data
Reports	<code>routes/reports.ts</code>	Date-range aggregated analytics endpoints
Reminders	<code>routes/reminders.ts</code>	Personal reminder CRUD
Notifications	<code>routes/notifications.ts</code>	Combined live alert feed

Two Express middleware functions — `authenticate` and `authorize(roles)` — are composed onto every protected route. `authenticate` verifies the JWT and attaches the decoded user to the request; `authorize` checks the attached user's role against an allow-list for that specific route. This means access control is enforced once, centrally, and consistently, rather than re-implemented inside each route handler.



13. Database Design

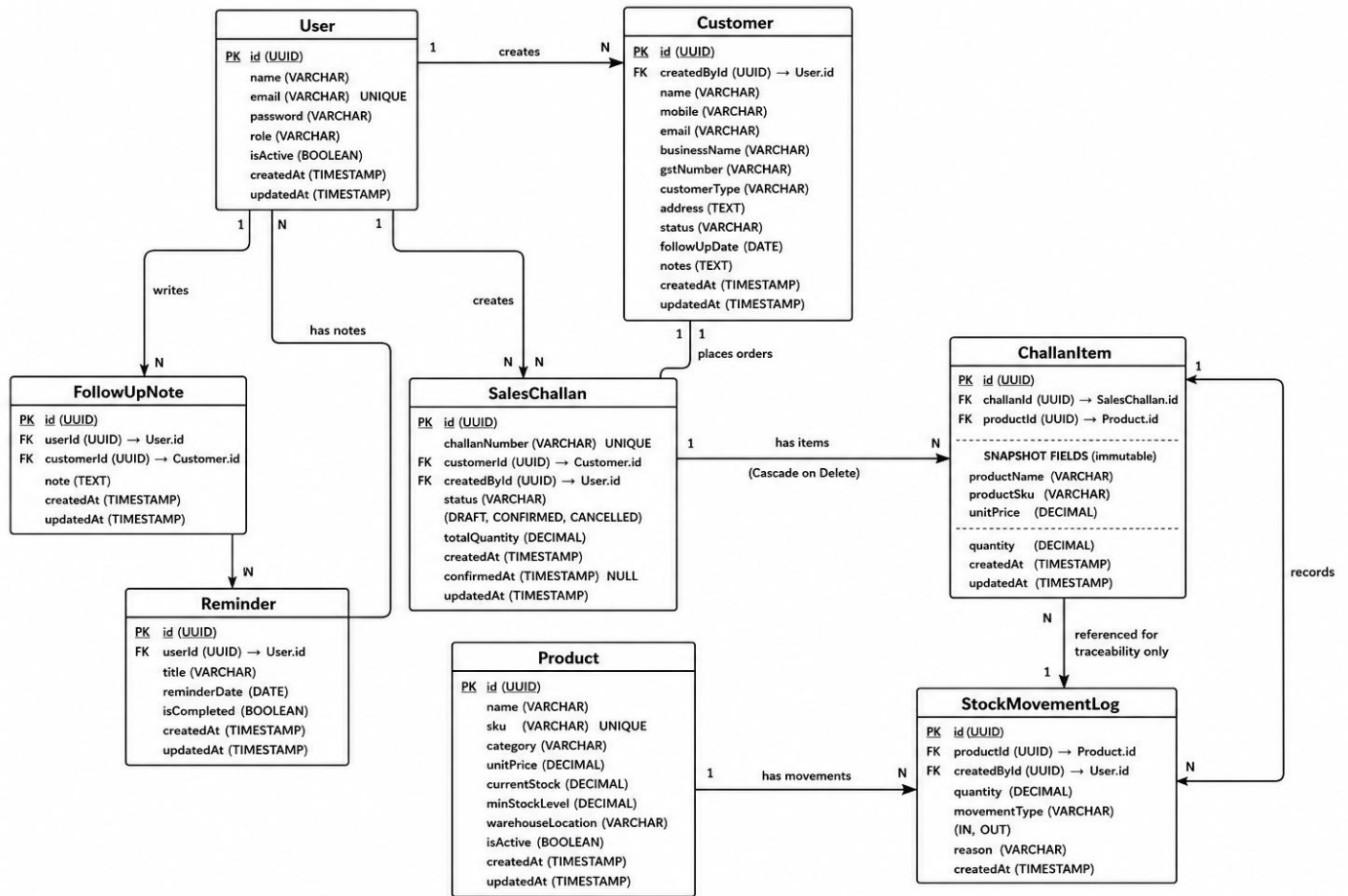
The schema is defined in Prisma and deployed to PostgreSQL via tracked migrations. Every table uses a UUID primary key and includes createdAt/updatedAt auditing fields where relevant.

Model	Purpose
User	System accounts; stores role and a bcrypt password hash
Customer	CRM records; owns a lifecycle status and follow-up date
FollowUpNote	Append-only, timestamped notes linked to a customer
Product	Inventory catalog; current stock and alert threshold
StockMovementLog	Immutable audit record of every stock change
SalesChallan	The sales document header: number, status, customer, totals
ChallanItem	Line items; stores a product snapshot, not a live reference
Reminder	Personal, per-user scheduled reminders

14. Schema / Entity Relationship Diagram

Key relationships across the schema:

- User (1) → (N) Customer — every customer records which user created it
- User (1) → (N) FollowUpNote — every note records its author
- User (1) → (N) StockMovementLog — every stock change records who made it
- User (1) → (N) SalesChallan — every challan records its creator
- User (1) → (N) Reminder — reminders are strictly personal to one user
- Customer (1) → (N) FollowUpNote — a customer's full contact history
- Customer (1) → (N) SalesChallan — a customer's full order history
- Product (1) → (N) StockMovementLog — a product's full audit trail
- Product (1) → (N) ChallanItem — for traceability/reporting only; display data comes from the snapshot fields on ChallanItem itself, not this relation
- SalesChallan (1) → (N) ChallanItem — the line items belonging to one challan (cascades on delete)



15. Authentication & RBAC

Authentication Flow

1. Client submits email + password to POST /api/auth/login.
2. Server looks up the user, compares the password against the stored bcrypt hash, and rejects with a generic error if either the email or password is wrong (no user-enumeration signal).
3. On success, the server signs a JWT containing the user's id and role.
4. The client attaches this token as a Bearer Authorization header on every subsequent request.
5. An authenticate middleware verifies the token's signature and expiry on every protected route before any handler logic runs.

Role-Based Access Control

Every protected route is wrapped with `authorize ([...roles])`, an explicit allow-list of which roles may call it. This enforcement happens on the server regardless of what the frontend UI shows or hides — a request from a Warehouse-role token to an Admin-only endpoint is rejected with 403 even if it bypasses the UI entirely.

Capability	Admin	Sales	Warehouse	Accounts
User Management	✓	—	—	—
View Customers	✓	✓	—	✓
Manage Customers / Follow-ups	✓	✓	—	—
View Products & Inventory	✓	✓	✓	✓
Adjust Inventory / View Logs	✓	—	✓	—
Create Sales Challans	✓	✓	—	—
View Sales Challans	✓	✓	✓	✓
Access Reports Dashboard	✓	—	—	✓
Export Challan PDF	✓	✓	✓	✓

16. Business Logic & Data Integrity

Preventing Negative Stock Under Concurrency

A naive implementation reads the current stock value, subtracts the requested quantity in application code, and writes the result back. Under concurrent requests, two processes can both read the same starting value before either writes back — the second write silently overwrites the first, a classic 'lost update.' OpsDesk avoids this entirely by using Prisma's atomic increment/decrement database operators, so the arithmetic happens as a single indivisible operation inside PostgreSQL itself, not in application memory.

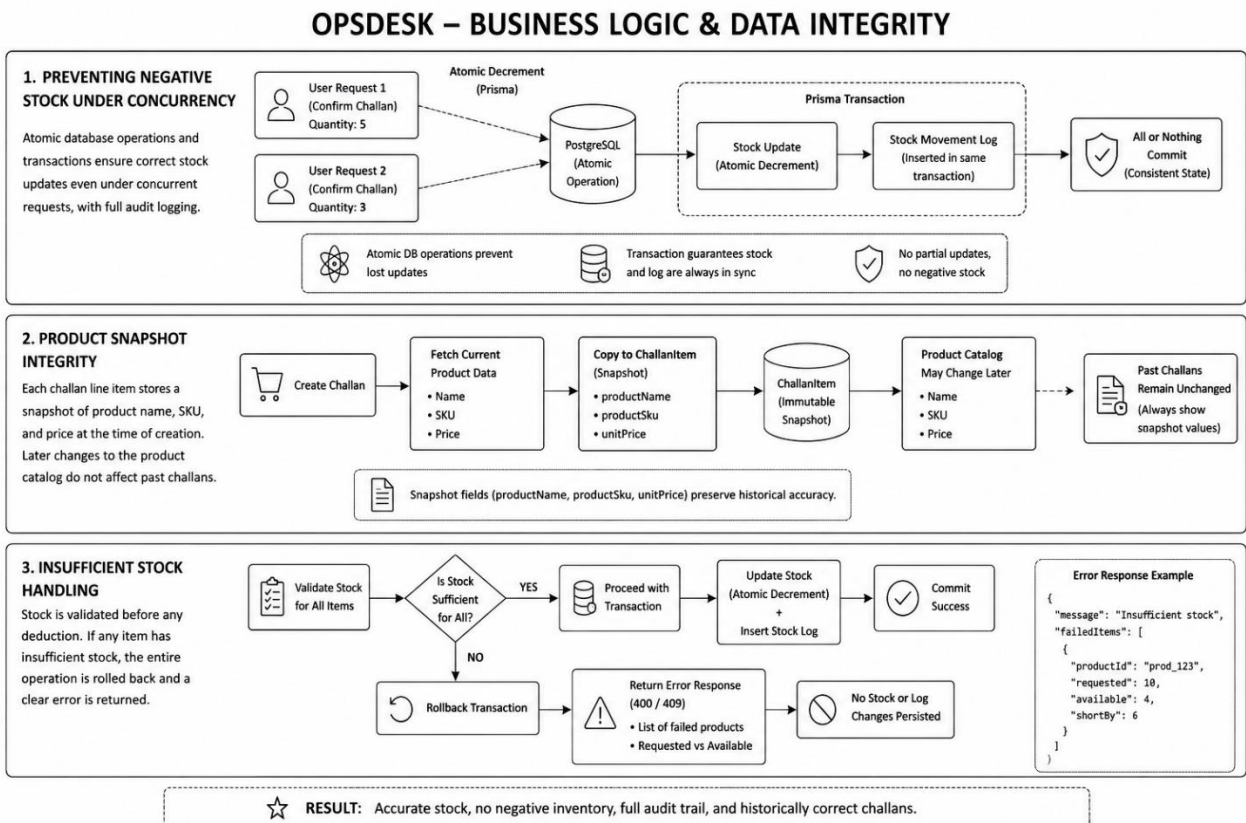
Every stock-changing action — challan confirmation, cancellation-with-restock, and manual adjustment — additionally runs inside a Prisma database transaction alongside its corresponding stock movement log entry, so a partial failure can never leave stock updated without a matching audit record, or vice versa.

Product Snapshot Integrity

When a challan is created, each line item copies the product's current name, SKU, and price directly into the ChallanItem row. Later edits to the product catalog have zero effect on previously issued challans — exactly how a real invoice or delivery note has to behave.

Insufficient Stock Handling

Stock sufficiency is validated before any deduction is attempted; if any line item requests more than is available, the entire transaction is rolled back and the API returns a structured 400/409-class response listing exactly which products failed and by how much, rather than a generic error or a partial, inconsistent update.

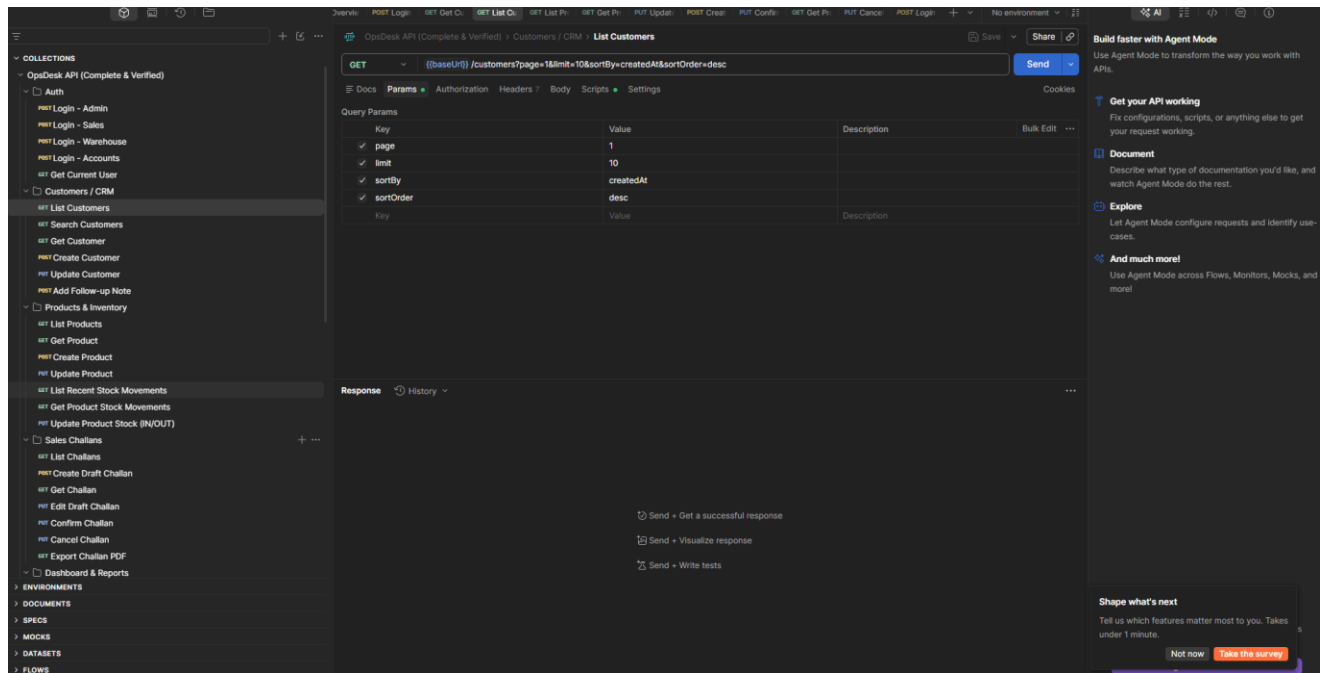


17. API Documentation

All endpoints are namespaced under `/api` and, aside from the auth routes and health check, require a valid Bearer token. A complete, ready-to-import Postman collection (`opsdesk_postman_collection.json`) is included in the repository and covers every endpoint below, along with RBAC-rejection and validation-failure scenarios.

Module	Endpoints
Auth	POST <code>/login</code> · GET <code>/me</code> · PATCH <code>/change-password</code>
Customers	GET <code>/</code> · GET <code>/:id</code> · POST <code>/</code> · PUT <code>/:id</code> · POST <code>/:id/notes</code>
Products	GET <code>/</code> · GET <code>/:id</code> · POST <code>/</code> · PUT <code>/:id</code> · PUT <code>/:id/stock</code> · GET <code>/:id/movements</code> · GET <code>/movements/recent</code>
Sales Challans	GET <code>/</code> · GET <code>/:id</code> · POST <code>/</code> · PUT <code>/:id</code> · PUT <code>/:id/confirm</code> · PUT <code>/:id/cancel</code> · GET <code>/:id/pdf</code>
Users	GET <code>/</code> · POST <code>/</code> · PUT <code>/:id</code>
Dashboard	GET <code>/</code>
Reports	GET <code>/sales-summary</code> · GET <code>/customer-breakdown</code> · GET <code>/inventory-health</code> · GET <code>/followup-compliance</code>
Reminders	GET <code>/</code> · POST <code>/</code> · PATCH <code>/:id</code> · DELETE <code>/:id</code>
Notifications	GET <code>/</code>

Every list endpoint (customers, products, challans) supports pagination and search/filter query parameters. Write endpoints validate their request body with a Zod schema before any database access, and respond with a consistent error shape (a top-level error field plus, for validation failures, per-field detail) across the whole API.



18. Security

Implemented Measures

- Passwords hashed with bcrypt before storage — plaintext passwords are never persisted.
- JWTs are verified server-side, with signature and expiry checked on every protected request.
- Role-based authorization is enforced in Express middleware, not just hidden in the UI.
- Every write endpoint validates its payload with Zod before it can reach the database.
- Helmet applies standard HTTP security headers on every response.
- The login endpoint is rate-limited to reduce brute-force attempts.
- CORS is restricted to an explicit, environment-configured allow-list of origins rather than left open.
- Error responses are sanitized in production — stack traces and internal error detail are logged server-side only, never returned to the client.
- No endpoint returns a user's password hash; user objects are selected/shaped explicitly rather than returned raw from the ORM.
- All secrets (JWT signing key, database connection string, CORS origin) are read from environment variables and are never committed to source control.

Known Trade-offs

This is an assignment-scoped MVP, and the following hardening decisions were made deliberately, with awareness of the production-grade alternative:

- JWTs are stored in localStorage on the client for implementation simplicity; an httpOnly, secure cookie would be the preferred approach for a hardened production deployment.
- Refresh-token rotation is not implemented — tokens are short-lived and re-issued on login only.
- The one-time admin-seeding endpoint used to bootstrap the first Admin account should be disabled or removed before any real production use.
- Very high-concurrency inventory workloads beyond this project's scale could benefit from additional row-level locking strategies on top of the atomic operations already in place.

19. Testing & Quality

Correctness was validated at three levels:

API-Level Testing

The included Postman collection exercises every endpoint under normal conditions, invalid-input conditions (expecting 400 with field-level errors), and unauthorized-role conditions (expecting 403), and has been cross-checked against the live backend's actual routes and validation schemas.

Business-Logic Testing

- Confirming a challan for more than available stock is rejected cleanly, with no partial stock deduction.
- Editing a product after a challan referencing it exists does not alter the already-issued challan's displayed values.
- Cancelling a confirmed challan with the restock option correctly restores stock and logs a corresponding IN movement.
- Simultaneous confirmation attempts against the same low-stock product were tested to confirm the atomic decrement prevents a lost-update/negative-stock outcome.

End-to-End Manual Testing

All four roles were logged in and walked through their permitted flows to confirm the frontend's role-aware navigation matches the backend's actual authorization rules, not just the visible UI.

20. Docker

A multi-stage Docker setup is included for fully reproducible local (or self-hosted) deployment, independent of the live Vercel/Render/Neon deployment.

Backend Image

- Builder stage compiles TypeScript and generates the Prisma Client.
- Runtime stage installs only production dependencies plus the Prisma CLI (required to run migrations at container start).
- Runs as the non-root node user rather than root.
- Applies pending Prisma migrations automatically before the server starts.

Compose Stack

Service	Port	Notes
postgres	5432	Data persisted in a named Docker volume
backend	4000	Express API, built from backend/Dockerfile
frontend	5175	Static build served via nginx

```
docker compose up --build
docker compose exec backend npx prisma migrate deploy
docker compose exec backend npx prisma db seed
```


21. CI/CD

The project's Docker configuration provides the foundation for continuous deployment: both the frontend and backend build into clean, reproducible images that can be triggered by any CI provider. Render and Vercel's native Git integration is used as the primary deployment mechanism — both platforms automatically rebuild and redeploy on a push to the main branch, which in practice provides continuous deployment without a hand-rolled pipeline.

A GitHub Actions workflow for linting/build verification is a natural next addition on top of this foundation and is noted under Future Enhancements; it was deprioritized in favor of hardening the core business logic and deployment reliability within the project's timeline.

22. Deployment Architecture

Layer	Platform	Detail
Frontend	Vercel	Framework: Vite · Root directory: frontend · VITE_API_URL injected at build time
Backend	Render	Docker build context: backend/ · Dockerfile-based deploy · reads PORT from the platform
Database	Neon	Managed PostgreSQL, connected via DATABASE_URL

A critical detail documented and handled correctly in this deployment: Vite inlines any VITE_-prefixed environment variable directly into the compiled JavaScript bundle at build time, not at runtime. VITE_API_URL is therefore supplied during the Vercel build step itself (pointing at the live Render backend URL), rather than as a runtime server variable, which would have no effect on an already-built static bundle.

23. Environment Variables

Backend (backend/.env)

Variable	Purpose
PORT	Port the Express server listens on (defaults to platform-provided PORT in production)
DATABASE_URL	PostgreSQL connection string (Neon in production)
JWT_SECRET	Signing secret for issued JWTs — the server refuses to start without it
CORS_ORIGIN	Comma-separated allow-list of permitted frontend origins — the server refuses to start without it

Frontend (build-time)

Variable	Purpose
VITE_API_URL	Base URL of the backend API — must be set at build time, not runtime

24. Local Setup

Backend

```
cd backend && npm install
cp .env.example .env # then fill in real values
npx prisma migrate dev --name init_postgres
npm run db:seed
npm run dev # http://localhost:3001
```

Frontend

```
cd frontend && npm install
npm run dev # http://localhost:5175
```

Or, the entire stack via Docker

```
docker compose up --build
```

25. Demo Walkthrough

The fastest way to experience the full operational lifecycle, using the live deployment:

1. Log in as Admin (admin@example.com) and explore the Reports analytics dashboard.
2. Log in as Sales (sales@example.com).
3. Navigate to Customers and create a new customer lead.
4. Navigate to Sales Challans and create a new challan for that customer, adding multiple line items.
5. Save the challan as Draft and observe its pipeline state.
6. Confirm the challan and observe the immediate stock deduction.
7. Click Export PDF on the confirmed challan's detail page to generate the branded PDF.
8. Log in as Warehouse (warehouse@example.com) and open Products & Inventory to see the stock movement log reflecting the deduction, with running IN/OUT/NET totals.

Role	Email	Password
Admin	admin@example.com	admin123
Sales	sales@example.com	sales123
Warehouse	warehouse@example.com	warehouse123
Accounts	accounts@example.com	accounts123

26. Screenshots

The following pages are recommended for screenshot capture when finalizing this document or the accompanying screen recording: the Landing page, the Login screen, the Dashboard, the Customer detail page, the Product detail page with its stock movement log, the Challan builder, the Challan detail page (including a deliberately-triggered insufficient-stock error), the exported PDF, and the Reports page.

Role	Email	Password
Admin	admin@example.com	admin123
Sales	sales@example.com	sales123
Warehouse	warehouse@example.com	warehouse123
Accounts	accounts@example.com	accounts123

Fig: Login Credentials

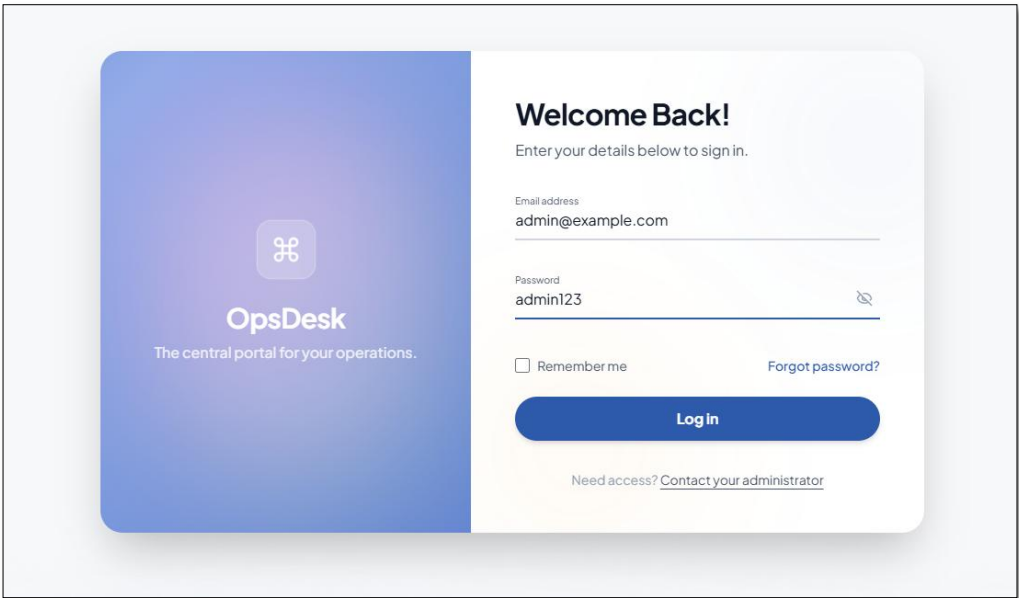


Fig: Login Page (Admin, sales, warehouse, Accounts)

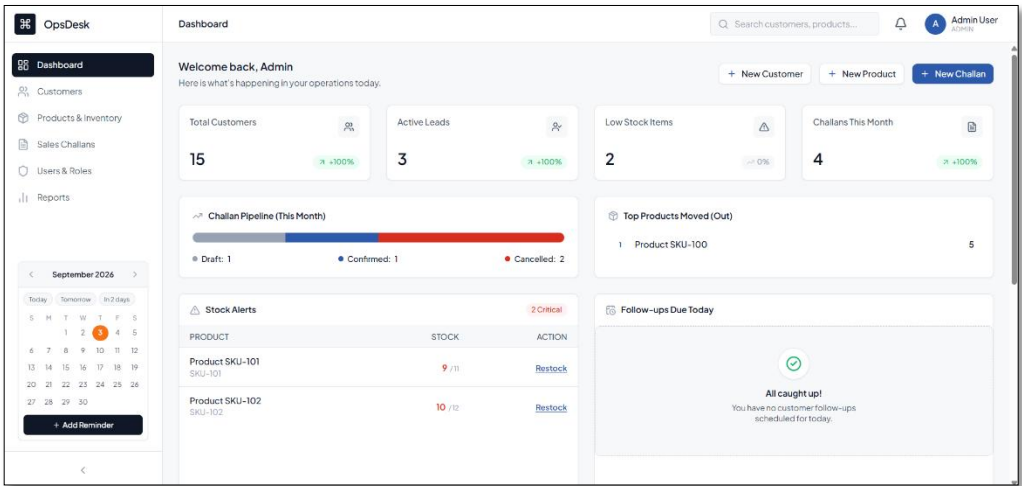


Fig: Dashboard Section

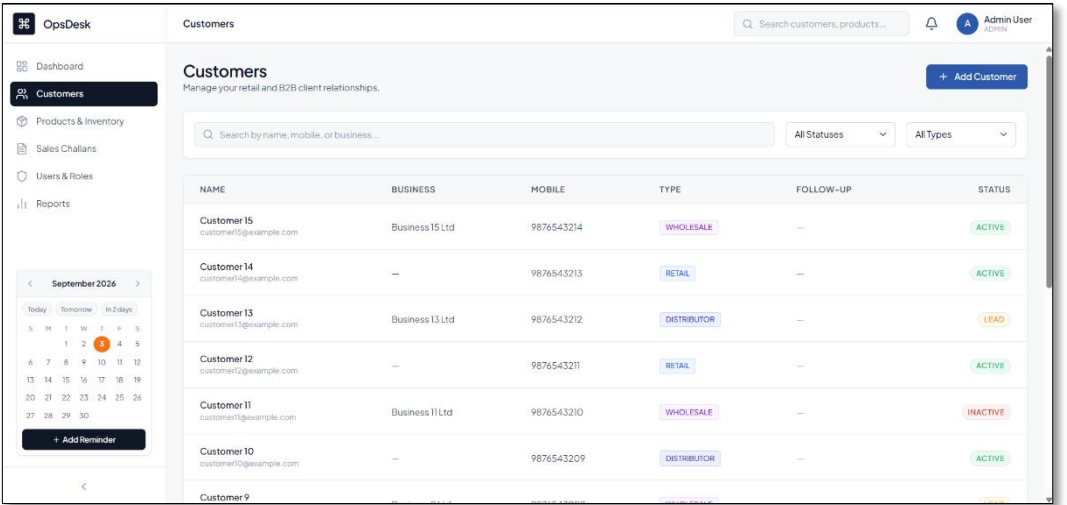


Fig: Customer Section

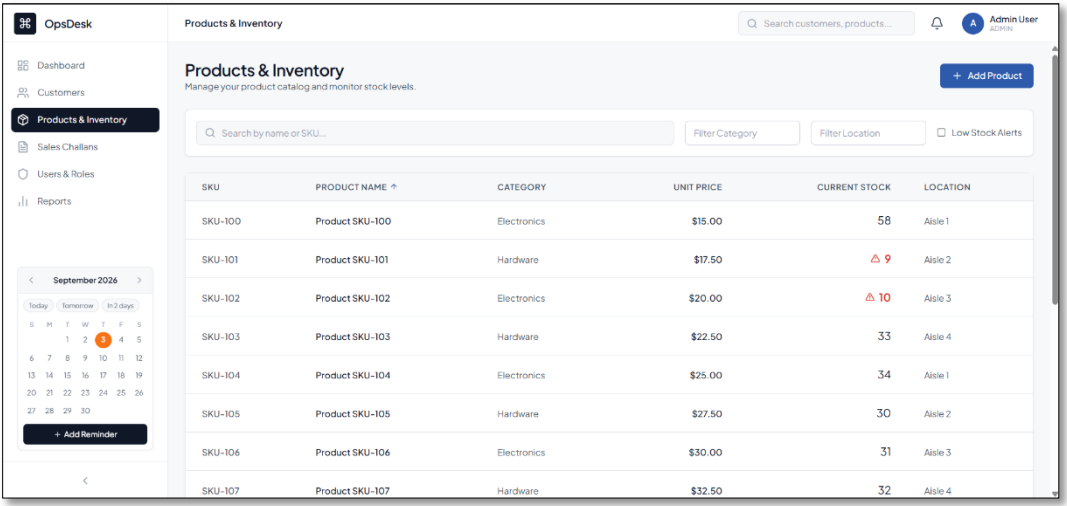


Fig: Product & Inventory

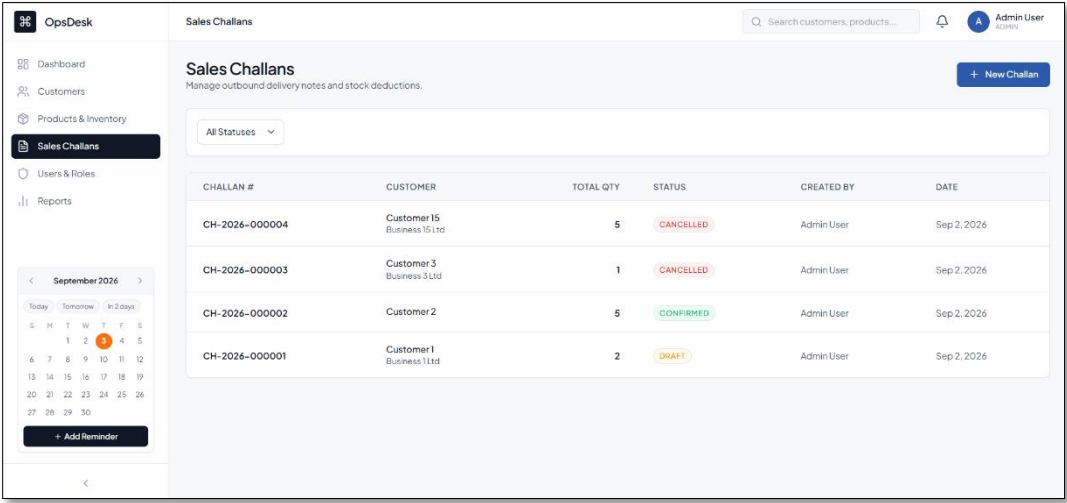


Fig: Sales Challans Section

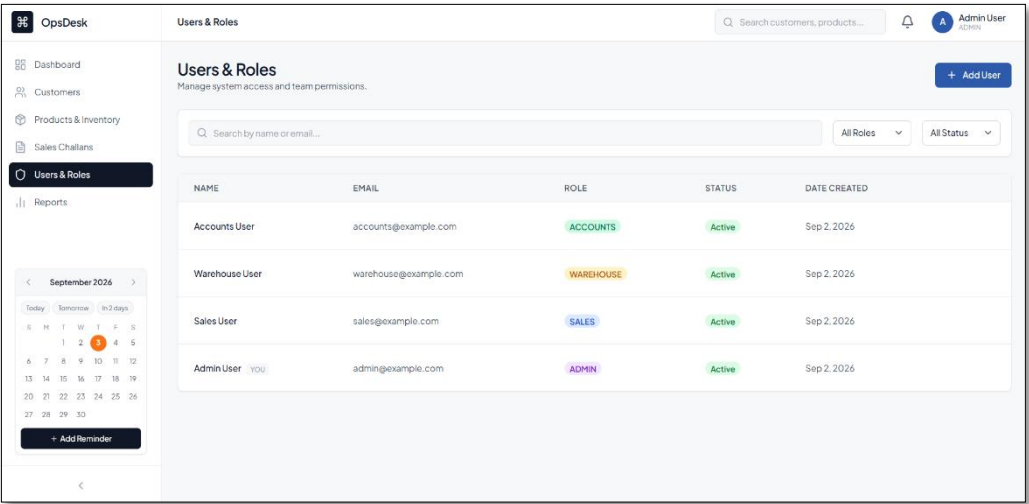


Fig: User & Roles Section

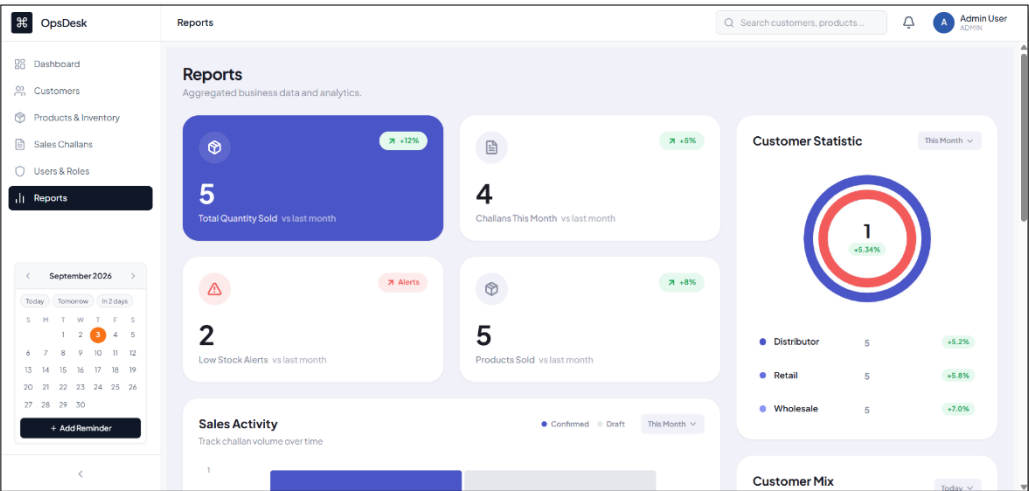


Fig: Reports Section

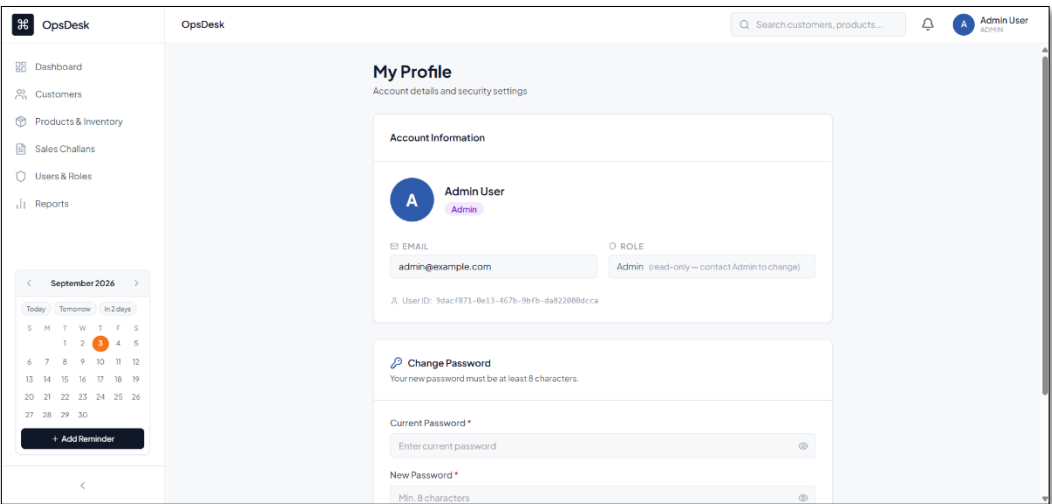


Fig: Profile Section (User, Change Password, Logout, Notification)

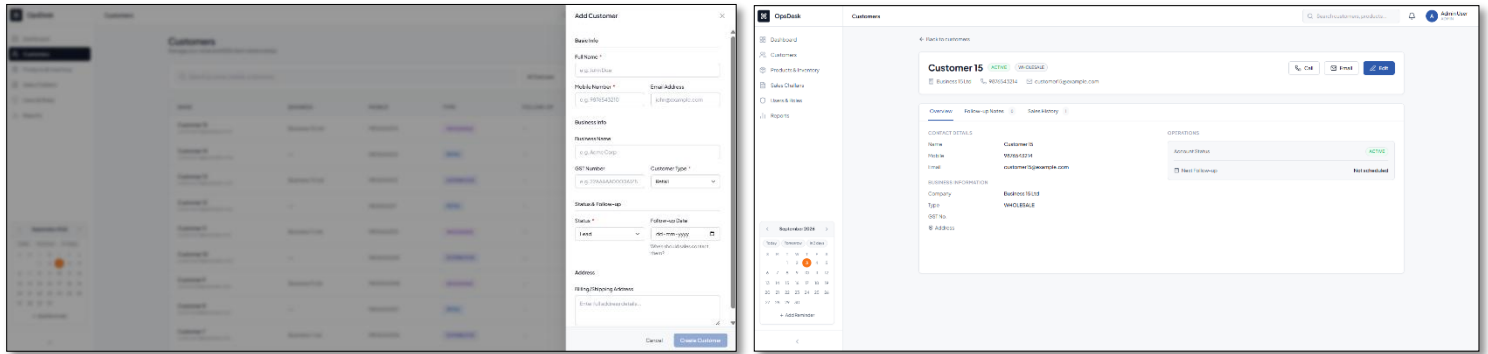


Fig: Add Customer & Customer interface (Customer Info, follow Up notes and history)

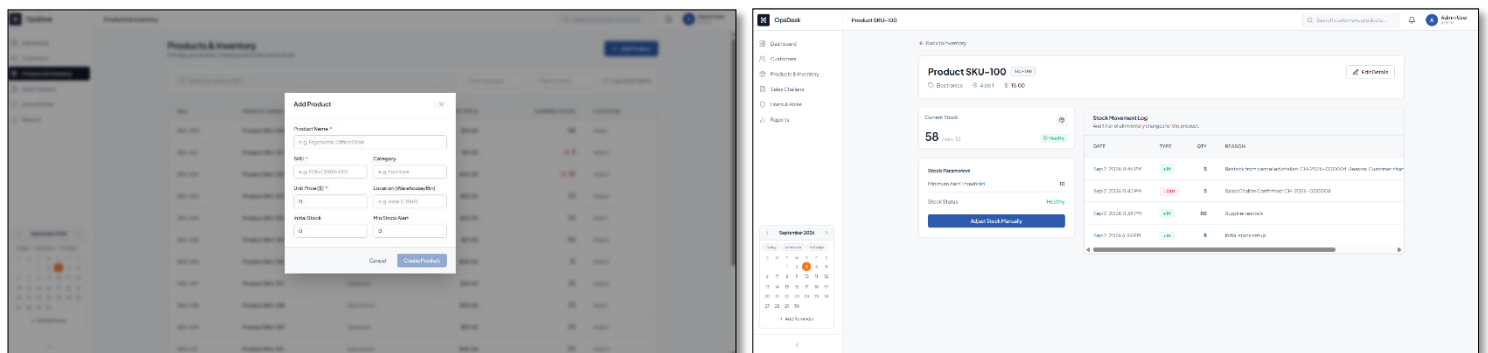


Fig: Add Product & Stock interface (Current stock, Stock Parameters, Adjust Stock Manually, Stock Movement Logs)

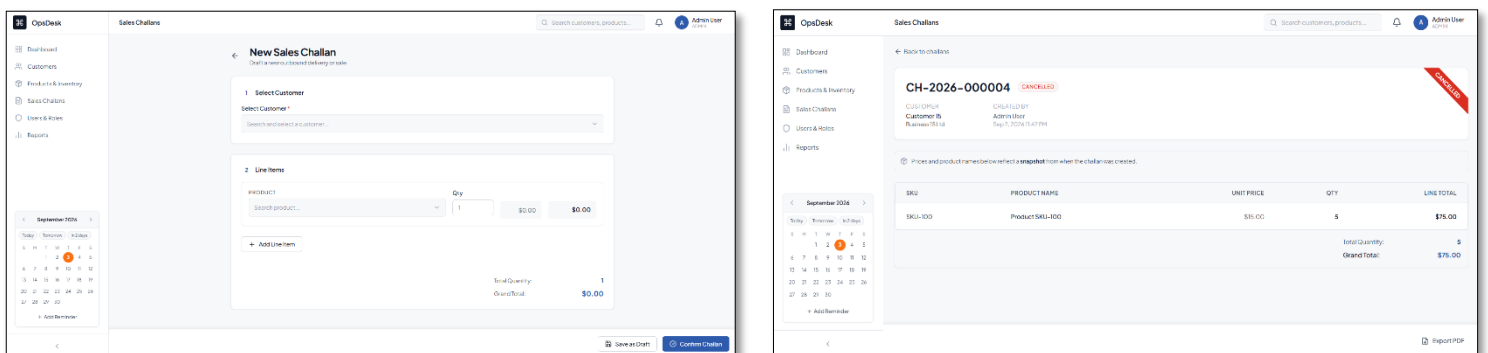


Fig: Add Challan & Challan Interface (Confirm, Cancel challan, Export PDF)

27. Assumptions

- The system is for internal company use only; there is no public customer-facing signup, and accounts are provisioned by an Administrator.
- A Sales user may both create and confirm challans directly, rather than requiring a separate approval step, consistent with a small-business workflow.
- GST number is optional on a customer record, reflecting that not every retail customer will have one.
- Purchase Orders and Invoices, mentioned only in passing in the business context, were treated as out of the required Core Modules and were intentionally not built as separate modules, in favor of fully completing and polishing the four modules the assignment explicitly requires.
- Free-tier hosting cold-start latency (the backend sleeping after inactivity) is an accepted characteristic of the deployment platform, not a defect.

28. Known Limitations

- JWTs are stored in browser localStorage rather than an httpOnly cookie, a deliberate simplicity trade-off for this project's scope (see Section 18).
- Refresh-token rotation is not implemented; sessions rely on short-lived tokens re-issued at login.
- A GitHub Actions CI workflow (lint/build verification) was not built; Vercel and Render's native Git-triggered deployments serve as the continuous-deployment mechanism instead.
- AWS deployment was not pursued, as it was explicitly listed as an optional bonus in the case study and free-tier alternatives (Vercel/Render/Neon) fully satisfy the deployment requirement without cost.
- Product image upload to S3 (a bonus item) was not implemented within the project timeline.
- The one-time admin-seed endpoint remains present and should be removed or access-restricted before any real production use beyond this evaluation.

29. Future Enhancements

- A GitHub Actions pipeline for automated linting, type-checking, and build verification on every pull request.
- httpOnly cookie-based token storage with refresh-token rotation.
- Product image upload via AWS S3, as outlined in the case study's bonus scope.
- Barcode-based inventory workflows for warehouse scanning.
- Deeper accounting/payment integrations to fully connect the Accounts role's workflow.
- Bulk import/export tooling for customers and products.
- More granular, per-user permission overrides beyond the four fixed roles.

30. Engineering Decisions & Trade-offs

Atomic Stock Operations Over Application-Level Math

Stock deduction/restoration uses Prisma's native increment/decrement operators inside a database transaction, rather than reading a value into application code and writing back a calculated result. This was a deliberate correction made specifically because the read-then-write pattern is vulnerable to lost updates under concurrent requests — a risk directly relevant to a multi-user sales environment where two reps could plausibly confirm challans for the same product at once.

Product Snapshotting Over Live References

Challan line items store copied product data rather than only a foreign key. This trades a small amount of data duplication for something a real business genuinely needs: a challan issued last month must never silently change because a product's price changed today.

Scope Discipline: Cutting Purchase Orders & Invoices

Purchase Orders and Invoices appear once in the case study's business-context paragraph but are never defined as Core Modules. Rather than build shallow, under-specified versions of both, the decision was made to drop them and instead fully complete, harden, and polish the four modules that are explicitly required — prioritizing depth over surface-level breadth.

Extending Scope Where It Adds Real Value

Conversely, Users & Roles and a Reports dashboard were added despite not being required, because the application already enforces four roles in practice (making user management a real operational gap without it) and because an ERP/CRM with no aggregate view of the business is a reasonable gap to close, given the time available.

31. Conclusion

OpsDesk was built to demonstrate more than checklist completion against the case study brief — the goal throughout was to build the kind of business logic and access control that a real wholesale operations team would actually depend on: stock that cannot be corrupted under concurrent use, historical sales documents that stay accurate, and permissions that are enforced where it actually matters, on the server.

The result is a live, publicly deployed system — not a local-only prototype — covering all four required core modules plus a meaningful set of additional functionality (Reports, Users & Roles, reminders, PDF export, and a fully custom UI), backed by a documented, honest account of what was built, what was deliberately scoped out, and what would come next in a longer-running version of this project.

End of Document